Create Rows - Use the Session - INSERT

Now that we have a database and a table, we can start adding data.

Here's a reminder of how the table would look like, this is the data we want to add:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

Create Table and Database

We will continue from where we left off in the last chapter.

This is the code we had to create the database and table, nothing new here:

Python 3.10+

```
from sqlmodel import Field, SQLModel, create_engine  ❷


class Hero(SQLModel, table=True):  ❸
    id: int | None = Field(default=None, primary_key=True)  ❹
    name: str  ❺
    secret_name: str  ❻
    age: int | None = None  ❼


sqlite_file_name = "database.db"  ❽
sqlite_url = f"sqlite:///{sqlite_file_name}"  ❾

engine = create_engine(sqlite_url, echo=True)  ❿


def create_db_and_tables():  ⓫
    SQLModel.metadata.create_all(engine)  ⓬

# More code here later 👈

if __name__ == "__main__":  ⓭
    create_db_and_tables()  ⓮
```

1. Import `Optional` from `typing` to declare fields that could be `None`.
2. Import the things we will need from `sqlmodel`: `Field`, `SQLModel`, `create_engine`.
3. Create the `Hero` model class, representing the `hero` table in the database.

   And also mark this class as a **table model** with `table=True`.
4. Create the `id` field:

   It could be `None` until the database assigns a value to it, so we annotate it with `Optional` (`int | None` in Python 3.10+).

   It is a **primary key**, so we use `Field()` and the argument `primary_key=True`.

5. Create the `name` field.

   It is required, so there's no default value, and it's not `Optional`.

6. Create the `secret_name` field.

   Also required.

7. Create the `age` field.

   It is not required, the default value is `None`.

   In the database, the default value will be `NULL`, the SQL equivalent of `None`.

   As this field could be `None` (and `NULL` in the database), we annotate it with `Optional` (`int | None` in Python 3.10+).

8. Write the name of the database file.

9. Use the name of the database file to create the database URL.

10. Create the engine using the URL.

    This doesn't create the database yet, no file or table is created at this point, only the **engine** object that will handle the connections with this specific database, and with specific support for SQLite (based on the URL).

11. Put the code that creates side effects in a function.

    In this case, only one line that creates the database file with the table.

12. Create all the tables that were automatically registered in `SQLModel.metadata`.

13. Add a main block, or "Top-level script environment".

    And put some logic to be executed when this is called directly with Python, as in:

    ```bash
    ```

    ...but that is not executed when importing something from this module, like:

    ```
    from app import Hero
    ```

14. In this main block, call the function that creates the database file and the table.

    This way when we call it with:

    ```bash
    ```

    ...it will create the database file and the table.

Now that we can create the database and the table, we will continue from this point and add more code on the same file to create the data.

Create Data with SQL

Before working with Python code, let's see how we can create data with SQL.

Let's say we want to insert the record/row for `Deadpond` into our database.

We can do this with the following SQL code:

```
INSERT INTO "hero" ("name", "secret_name")
VALUES ("Deadpond", "Dive Wilson");
```

It means, more or less:

> Hey SQL database 👋, please `INSERT` something (create a record/row) `INTO` the table `"hero"`.
>
> I want you to insert a row with some values in these specific columns:
>
> - `"name"`
> - `"secret_name"`
>
> And the values I want you to put in these columns are:
>
> - `"Deadpond"`
> - `"Dive Wilson"`

Try it in DB Explorer for SQLite

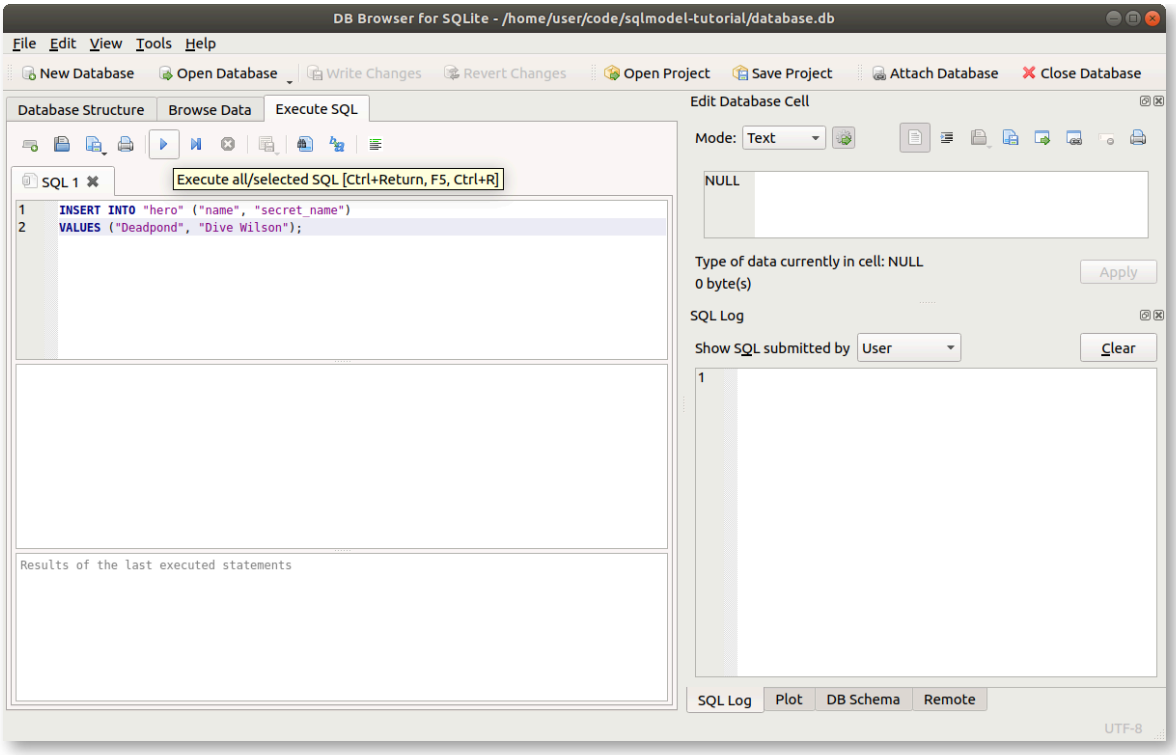
You can try that SQL statement in **DB Explorer for SQLite**.

Make sure to open the same database we already created by clicking `Open Database` and selecting the same `database.db` file.

> **Tip**
>
> If you don't have that `database.db` file with the table `hero`, you can re-create it by running the Python program at the top. 👆

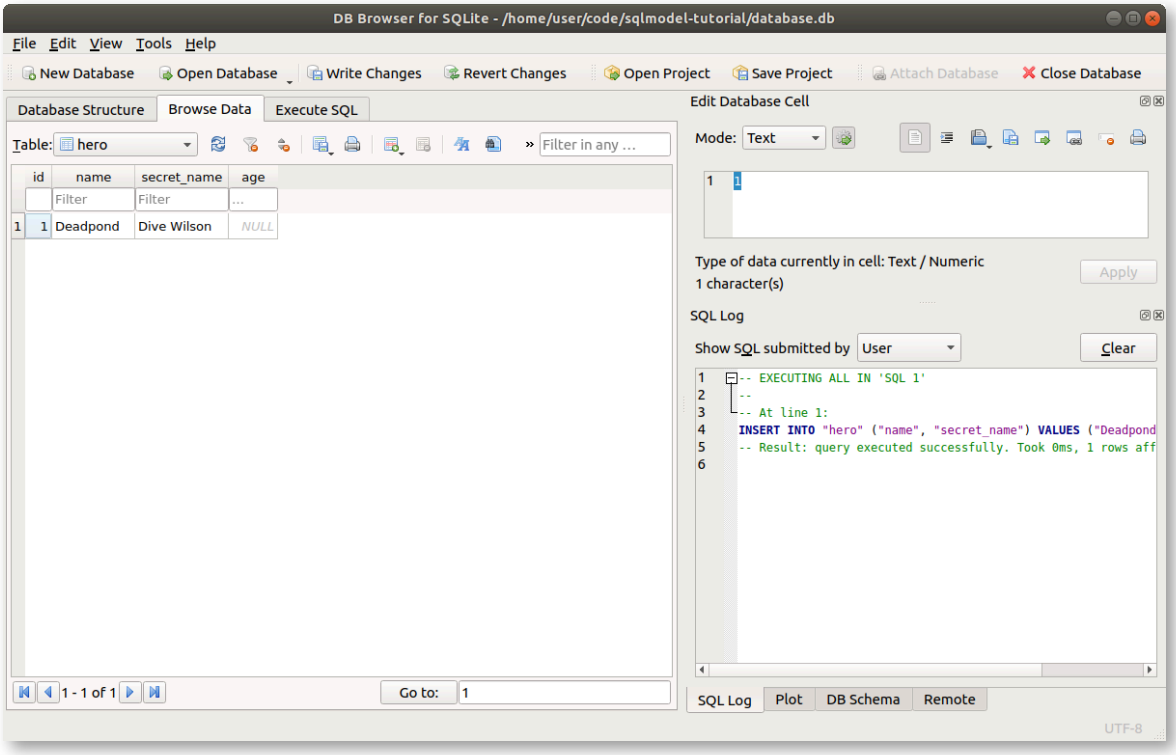
Then go to the `Execute SQL` tab and copy the SQL from above.

It would look like this:



Click the "Execute all" button.

Then you can go to the `Browse Data` tab, and you will see your newly created record/row:



Data in a Database and Data in Code

When working with a database (SQL or any other type) in a programming language, we will always have some data **in memory**, in objects and variables we create in our code, and there will be some data **in the database**.

We are constantly **getting** *some* of the data from the database and putting it in memory, in variables.

The same way, we are constantly **creating** variables and objects with data in our code, that we then want to save in the database, so we **send** it somehow.

In some cases, we can even create some data in memory and then change it and update it before saving it in the database.

We might even decide with some logic in the code that we no longer want to save the data in the database, and then just remove it. 🔥 And we only handled that data in memory, without sending it back and forth to the database.

SQLModel does all it can (actually via SQLAlchemy) to make this interaction as simple, intuitive, and familiar or "close to programming" as possible. ✨

But that division of the two places where some data might be at each moment in time (in memory or in the database) is always there. And it's important for you to have it in mind. 🤓

Create Data with Python and SQLModel

Now let's create that same row in Python.

First, remove that file `database.db` so we can start from a clean slate.

Because we have Python code executing with data in memory, and the database is an independent system (an external SQLite file, or an external database server), we need to perform two steps:

- create the data in Python, in memory (in a variable)
- save/send the data to the database

Create a Model Instance

Let's start with the first step, create the data in memory.

We already created a class `Hero` that represents the `hero` table in the database.

Each instance we create will represent the data in a row in the database.

So, the first step is to simply create an instance of `Hero`.

We'll create 3 right away, for the 3 heroes:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Tip

The code above in this file (the omitted code) is just the same code that you see at the top of this chapter.

The same code we used before to create the `Hero` model.

We are putting that in a function `create_heroes()`, to call it later once we finish it.

If you are trying the code interactively, you could also write that directly.

Create a Session

Up to now, we have only used the **engine** to interact with the database.

The **engine** is that single object that we share with all the code, and that is in charge of communicating with the database, handling the connections (when using a server database like PostgreSQL or MySQL), etc.

But when working with **SQLModel** you will mostly use another tool that sits on top, the **Session**.

In contrast to the **engine** that is one for the whole application, we create a new **session** for each group of operations with the database that belong together.

In fact, the **session** needs and uses an **engine**.

For example, if we have a web application, we would normally have a single **session** per request.

We would re-use the same **engine** in all the code, everywhere in the application (shared by all the requests). But for each request, we would create and use a new **session**. And once the request is done, we would close the session.

The first step is to import the `Session` class:

Python 3.10+

```
from sqlmodel import Field, Session, SQLModel, create_engine

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    session = Session(engine)

    session.add(hero_1)
    session.add(hero_2)
    session.add(hero_3)

    session.commit()

    session.close()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Then we can create a new session:

Python 3.10+

```
# Code above omitted 👆

def create_heroes():
```

```
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    session = Session(engine)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    session = Session(engine)

    session.add(hero_1)
    session.add(hero_2)
    session.add(hero_3)

    session.commit()

    session.close()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

The new `Session` takes an `engine` as a parameter. And it will use the **engine** underneath.

Tip

We will see a better way to create a **session** using a `with` block later.

Add Model Instances to the Session

Now that we have some hero model instances (some objects in memory) and a **session**, the next step is to add them to the session:

Python 3.10+

```
# Code above omitted 👆

def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    session = Session(engine)

    session.add(hero_1)
    session.add(hero_2)
    session.add(hero_3)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    session = Session(engine)

    session.add(hero_1)
    session.add(hero_2)
    session.add(hero_3)

    session.commit()

    session.close()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

By this point, our heroes are *not* stored in the database yet.

And this is one of the cases where having a **session** independent of an **engine** makes sense.

The session is holding in memory all the objects that should be saved in the database later.

And once we are ready, we can **commit** those changes, and then the **session** will use the **engine** underneath to save all the data by sending the appropriate SQL to the database, and that way it will create all the rows. All in a single batch.

This makes the interactions with the database more efficient (plus some extra benefits).

Technical Details

The session will create a new transaction and execute all the SQL code in that transaction.

This ensures that the data is saved in a single batch, and that it will all succeed or all fail, but it won't leave the database in a broken state.

Commit the Session Changes

Now that we have the heroes in the **session** and that we are ready to save all that to the database, we can **commit** the changes:

Python 3.10+

```
# Code above omitted 👆

def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    session = Session(engine)

    session.add(hero_1)
    session.add(hero_2)
    session.add(hero_3)

    session.commit()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    session = Session(engine)

    session.add(hero_1)
    session.add(hero_2)
    session.add(hero_3)

    session.commit()

    session.close()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Once this line is executed, the **session** will use the **engine** to save all the data in the database by sending the corresponding SQL.

Create Heroes as a Script

The function to create the heroes is now ready.

Now we just need to make sure to call it when we run this program with Python directly.

We already had a main block like:

```python
if __name__ == "__main__":
    create_db_and_tables()
```

We could add the new function there, as:

```python
if __name__ == "__main__":
    create_db_and_tables()
    create_heroes()
```

But to keep things a bit more organized, let's instead create a new function `main()` that will contain all the code that should be executed when called as an independent script, and we can put there the previous function `create_db_and_tables()`, and add the new function `create_heroes()`:

Python 3.10+

```
# Code above omitted 👆

def main():
    create_db_and_tables()
    create_heroes()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

And then we can call that single `main()` function from that main block:

Python 3.10+

```
# Code above omitted 👆

def main():
    create_db_and_tables()
    create_heroes()
```

```python
if __name__ == "__main__":
    main()
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

By having everything that should happen when called as a script in a single function, we can easily add more code later on.

And some other code could also import and use this same `main()` function if it was necessary.

Run the Script

Now we can run our program as a script from the console.

Because we created the **engine** with `echo=True`, it will print out all the SQL code that it is executing:

```bash
```

If you have ever used Git, this works very similarly.

We use `session.add()` to add new objects (model instances) to the session (similar to `git add`).

And that ends up in a group of data ready to be saved, but not saved yet.

We can make more modifications, add more objects, etc.

And once we are ready, we can **commit** all the changes in a single step (similar to `git commit`).

Close the Session

The **session** holds some resources, like connections from the engine.

So once we are done with the session, we should **close** it to make it release those resources and finish its cleanup:

Python 3.10+

```
# Code above omitted 👆

def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    session = Session(engine)

    session.add(hero_1)
    session.add(hero_2)
    session.add(hero_3)

    session.commit()

    session.close()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    session = Session(engine)

    session.add(hero_1)
    session.add(hero_2)
    session.add(hero_3)

    session.commit()

    session.close()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

But what happens if we forget to close the session?

Or if there's an exception in the code and it never reaches the `session.close()`?

For that, there's a better way to create and close the session, using a `with` block. 👇

A Session in a `with` Block

It's good to know how the `Session` works and how to create and close it manually. It might be useful if, for example, you want to explore the code in an interactive session (for example with Jupyter).

But there's a better way to handle the session, using a `with` block:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
```

```
    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

This is the same as creating the session manually and then manually closing it. But here, using a `with` block, it will be automatically created when **starting** the `with` block and assigned to the variable `session`, and it will be automatically closed after the `with` block is **finished**.

And it will work even if there's an exception in the code. 😎

Review All the Code

Let's give this whole file a final look. 🔍

You already know all of the first part for creating the `Hero` model class, the **engine**, and creating the database and table.

Let's focus on the new code:

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():  1
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")  2
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:  3
        session.add(hero_1)  4
        session.add(hero_2)
        session.add(hero_3)

        session.commit()  5
    6


def main():  7
    create_db_and_tables()  8
    create_heroes()  9


if __name__ == "__main__":  10
    main()  11
```

1. We use a function `create_heroes()` to put this logic together.
2. Create each of the objects/instances of the `Hero` model.

   Each of them represents the data for one row.
3. Use a `with` block to create a `Session` using the `engine`.

   The new **sesion** will be assigned to the variable `session`.

   And it will be automatically closed when the `with` block is finished.
4. Add each of the objects/instances to the **session**.

   Each of these objects represents a row in the database.

   They are all waiting there in the session to be saved.
5. **Commit** the changes to the database.

   This will actually send the data to the database.

   It will start a transaction automatically and save all the data in a single batch.
6. By this point, after the `with` block is finished, the **session** is automatically closed.
7. We have a `main()` function with all the code that should be executed when the program is called as a **script from the console**.

   That way we can add more code later to this function.

   We then put this function `main()` in the main block below.

And as it is a single function, other Python files could **import it** and call it directly.

8 In this `main()` function, we are also creating the database and the tables.

In the previous version, this function was called directly in the main block.

But now it is just called in the `main()` function.

9 And now we are also creating the heroes in this `main()` function.

10 We still have a main block to execute some code when the program is run as a script from the command line, like:

```bash
```

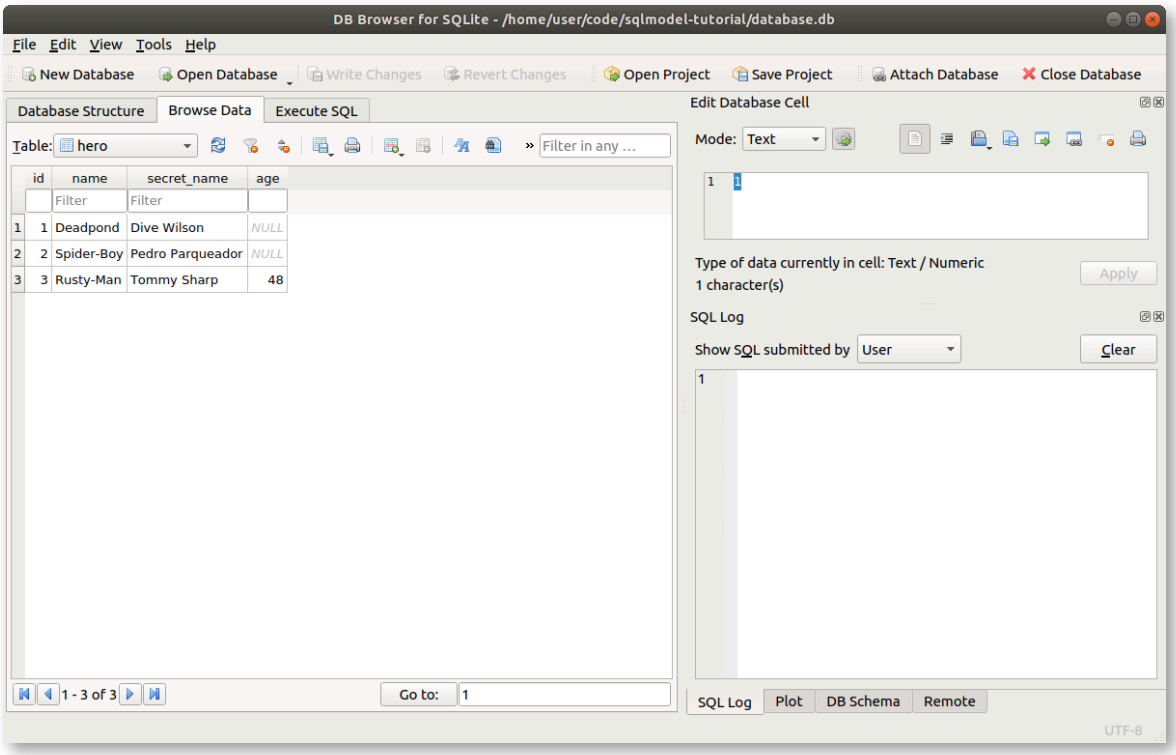
11 There's a single `main()` function now that contains all the code that should be executed when running the program from the console.

So this is all we need to have in the main block. Just call the `main()` function.

> **Tip**
>
> Review what each line does by clicking each number bubble in the code. 👆

You can now put it in a `app.py` file and run it with Python. And you will see an output like the one shown above.

After that, if you open the database with **DB Browser for SQLite**, you will see the data you just created in the `Browse Data` tab:



What's Next

Now you know how to add rows to the database. 🎉

Now is a good time to understand better why the `id` field **can't be** `NULL` on the database because it's a **primary key**, but actually **can be** `None` in the Python code.

I'll tell you about that in the next chapter. 🚀